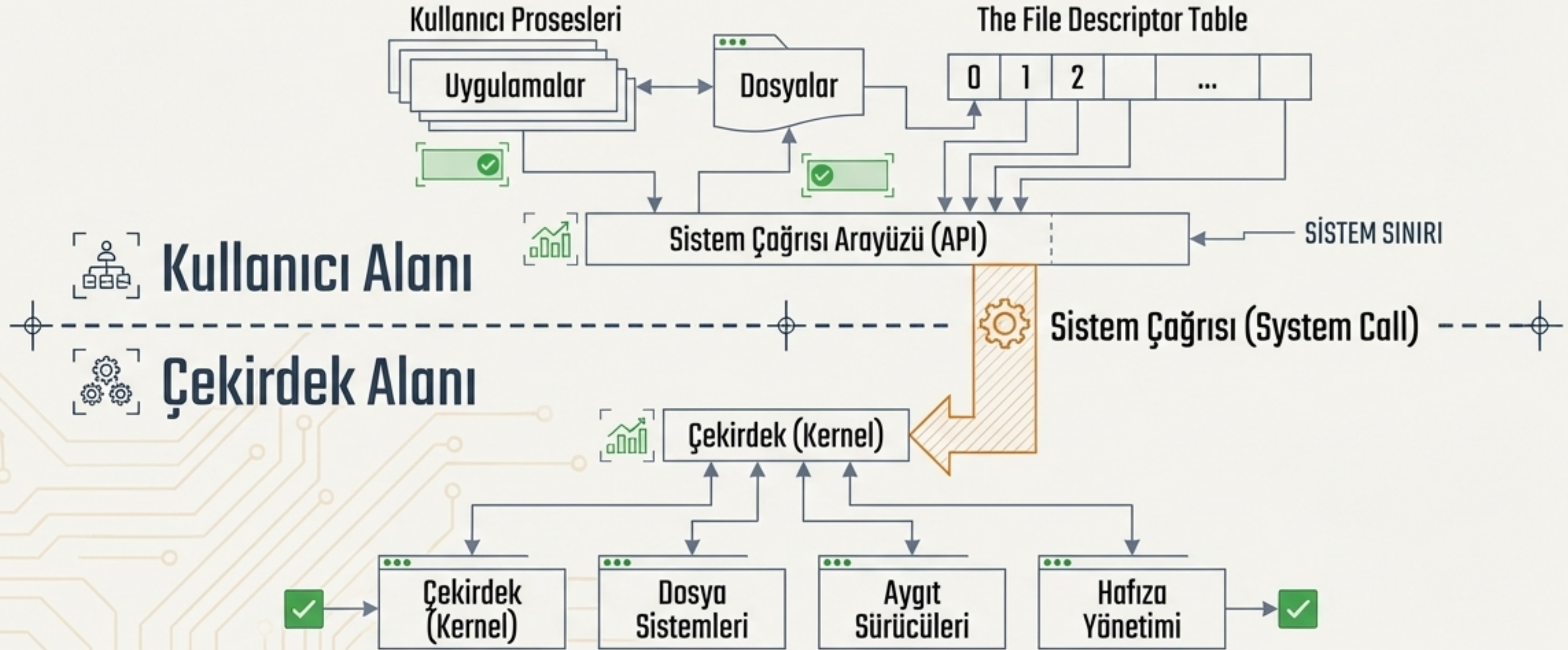


Kullanıcı ve Çekirdek Arasındaki Köprü

Unix Sistem Programlama: Dosyalar, Prosesler ve Sistem Çağrılar

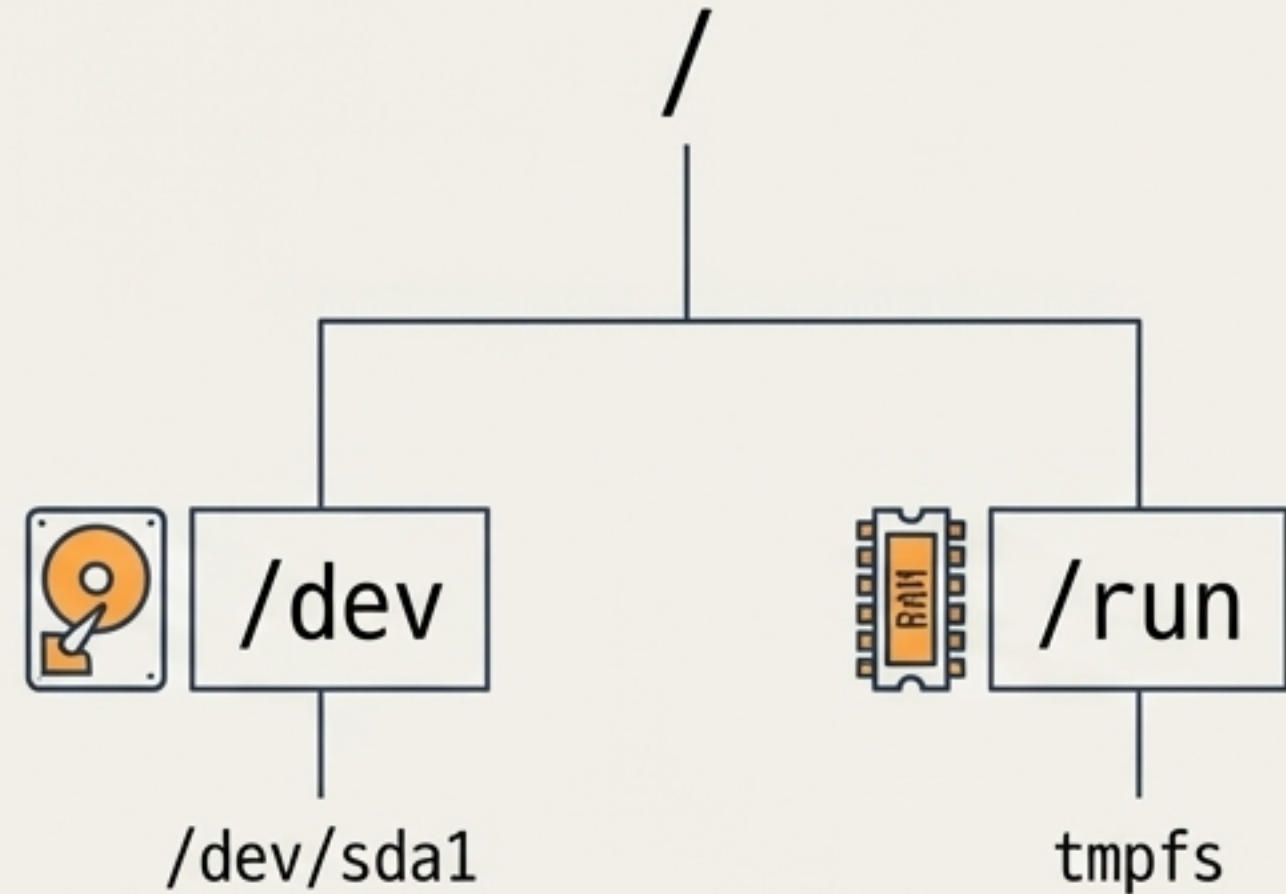


Dosya Sistemi: Hiyerarşik Düzen

Unix'te her şey kök (root) dizin / ile başlar. **df** komutu, makinedeki disk bölümlerinin (/dev/sda1) veya RAM tabanlı sanal sistemlerin ('tmpfs') bu ağaca nasıl bağlandığını gösterir.

```
abdullah@abdullah-VirtualBox:~$ df
```

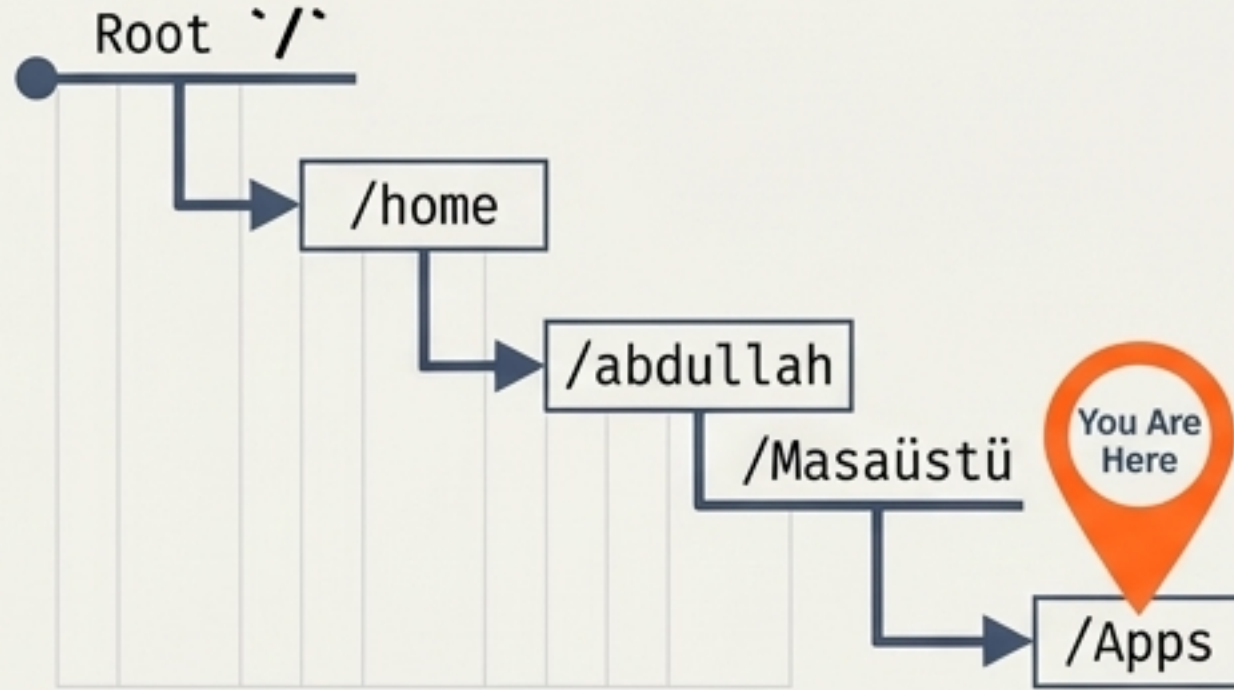
	1K-blok	Bağlanılan yer	Dolu	Boş	Kull%	
udev	1986140	...	/dev	1986140	0	0%
tmpfs	403916	...	/run	403916	1540	2%
/dev/sda1	20464208	...	/	20464208	0	20%
tmpfs	2019560	...	/dev/shm	2019560	7000	5%
tmpfs	5120	...	/run/lock	5120	0	0%
tmpfs	2019560	...	/sys/fs/cgroup	2019560	2000	5%



Adlar, Yollar ve Yön Bulma

Mutlak Yol Adı

Daima `/` ile başlar. Eğik çizgi dizisidir.

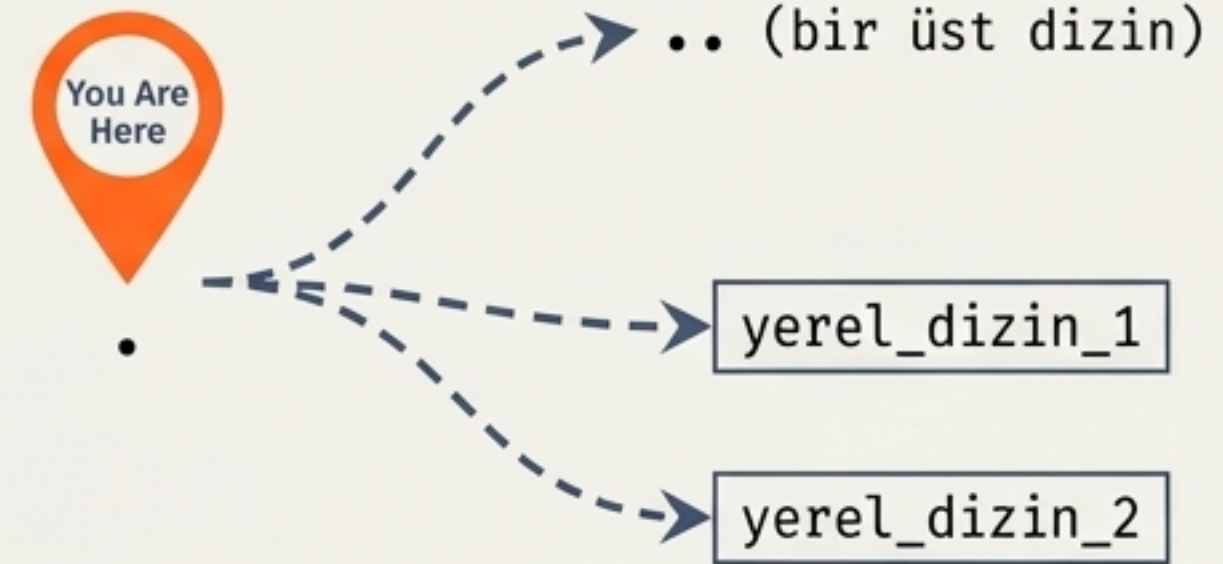


Tam konumu belirtir.

```
Root '/'  
/home/  
/yere1_dizin_1  
/Masaüstü/  
/yere1_dizin_1  
/yere1_dizin_2  
/Apps/
```

Göreceli Yol Adı

`/` ile başlamaz.
Mevcut "Çalışma Dizini"ne (`.`) bağlıdır.



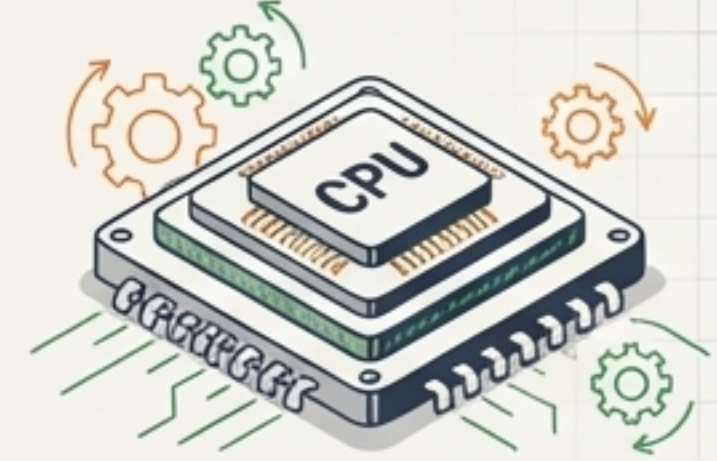
```
abduallah@abduallah-VirtualBox:~/Masaüstü/Apps$ pwd  
/home/abduallah/Masaüstü/Apps  
abduallah@abduallah-VirtualBox:~/Masaüstü/Apps$ cd ..  
abduallah@abduallah-VirtualBox:~/Masaüstü$ pwd  
/home/abduallah/Masaüstü  
abduallah@abduallah-VirtualBox:~/Masaüstü$ cd ..  
abduallah@abduallah-VirtualBox:~$ pwd  
/home/abduallah
```


Vatandaşları Tanımak: Program vs. Proses

Program



Proses



Durum (**State**)

Statik, cansız dosya.

Yürütülmekte olan, aktif örnek.

Konum
(**Location**)

Diskte yatar (``/bin/ls``).

RAM'de çalışır.

Kimlik
(**Identifier**)

Dosya adı.

İşletim sistemi tarafından atanan
PID (Proses ID).

Prosesleri Gözlemlemek

`ps x` komutu sistemdeki aktif işlemleri listeler. Vim gibi bir düzenleyiciyi her açtığınızda, diskteki tek bir programdan bellekte tamamen yeni ve benzersiz bir proses (farklı bir PID ile) doğar.

```
abduallah@abduallah-VirtualBox:~$ ps x
```

PID	TTY	STAT	TIME	COMMAND
1	pts/0	S+	0:00	/usr/lib/systemd/systemd
367	pts/	S+	0:00	/usr/lib/xorg/Xorg -- gnome-shell, -is --eletikor- 0x00000000 0x00000000,
1222	pts/0	S+	0:00	/usr/bin/gnome-shell --gnome-shell
5923	pts/0	S	0:00	/bin/bash
5928	pts/0	S+	0:00	/usr/bin/vim
5942	pts/1	S+	0:00	/usr/lib/systemd/Xorg
5947	pts/1	S+	0:00	/usr/bin/gnome-shell
5963	pts/1	S+	0:00	/usr/bin/gnome-shell
5966	pts/1	S+	0:00	/usr/lib/xorg/Xorg -- gnomemvnrk
5977	pts/1	S+	0:00	/usr/lib/systemd/systemd -- denmat2
5987	pts/1	S+	0:00	/usr/bin/vim
5928	pts/0	S+	0:00	vim /p1.c
6002	pts/1	S+	0:00	vim /anotherfile.txt
5928	pts/0	S+	0:00	vim /p1.c
6002	pts/1	S+	0:00	vim /anotherfile.txt

Sinyaller: Kontrol Mekanizması

Sinyal, bir prosese gönderilen sistem düzeyinde bir kesmedir. Sinyal işleyiciler (Signal Handlers) bu kesmeleri yönetir. Bir işlemi durdurabilir (fg ile devam ettirilebilir) veya kalıcı olarak sonlandırabilirsiniz.

C Code

```
/* This program prints out a counter indefinitely.
   You stop it temporarily by sending a SIGSTOP signal
   to the program. You can run it again in the background
   with `fg`. You can terminate it with a SIGINT signal,
   which is sent by typing <CNTRL-C>. */
```

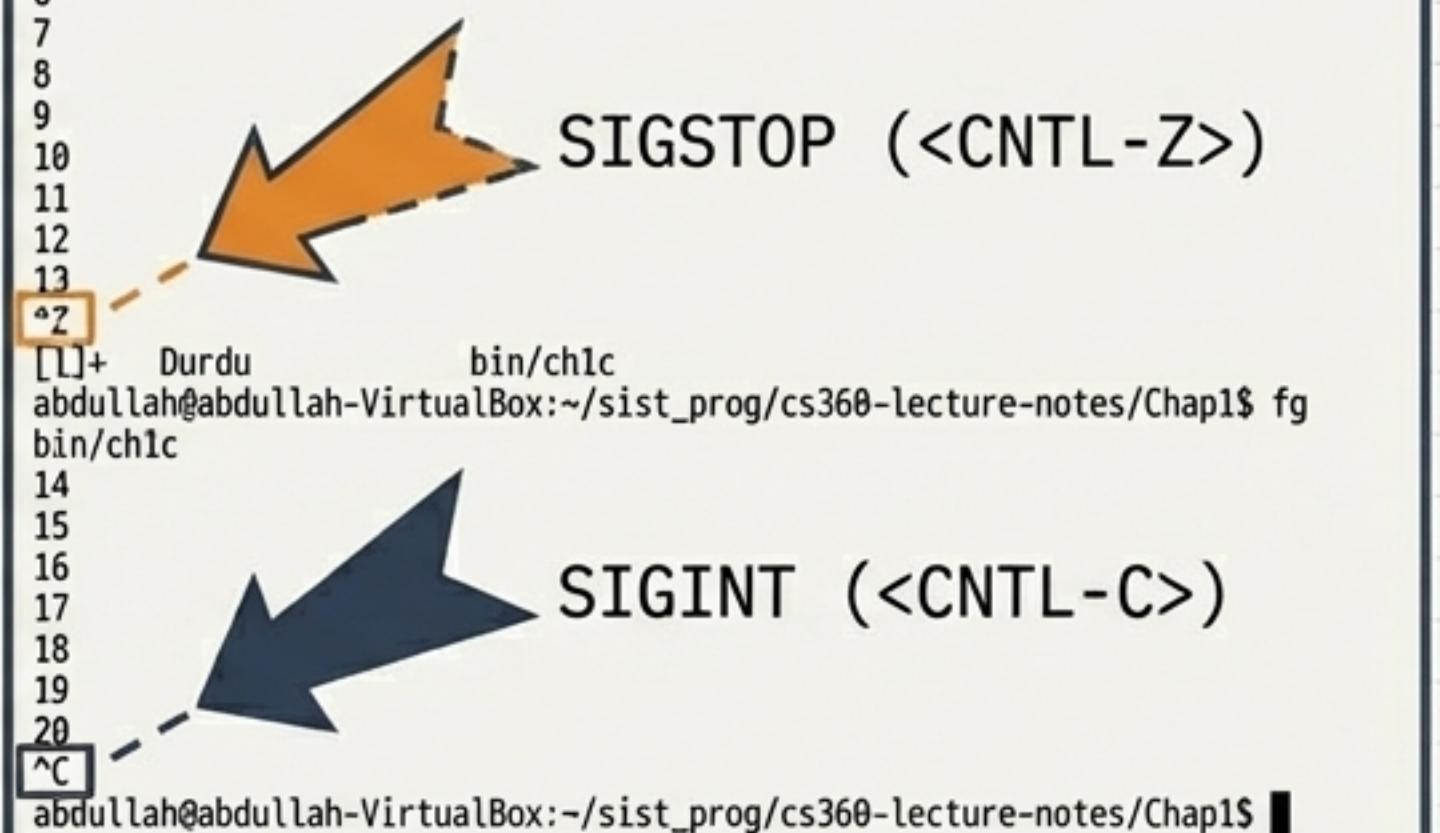
```
#include <stdio.h>
#include <unistd.h>

int main()
{
    int i;

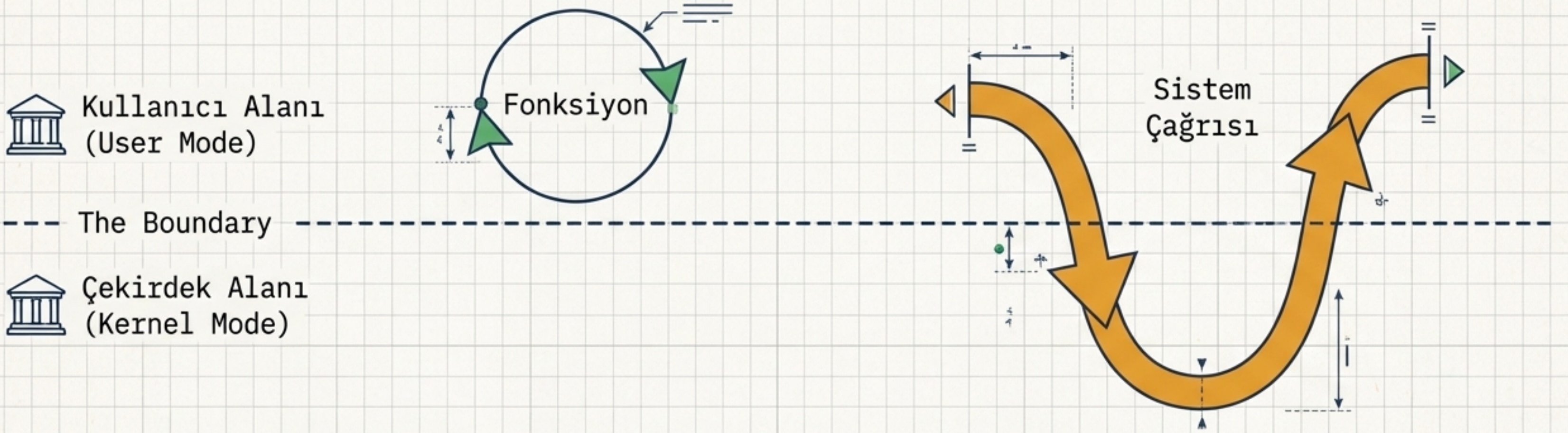
    i = 0;
    while (1) {
        i++;
        printf("%d\n", i);
        fflush(stdout);
        sleep(1);
    }
    return 0;
}
```

Terminal Output

```
abdullah@abdullah-VirtualBox:~/sist_prog/cs360-lecture-notes/Chap1$ bin/ch1c
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
^Z
[1]+  Durdu bin/ch1c
abdullah@abdullah-VirtualBox:~/sist_prog/cs360-lecture-notes/Chap1$ fg
bin/ch1c
14
15
16
17
18
19
20
^C
abdullah@abdullah-VirtualBox:~/sist_prog/cs360-lecture-notes/Chap1$
```



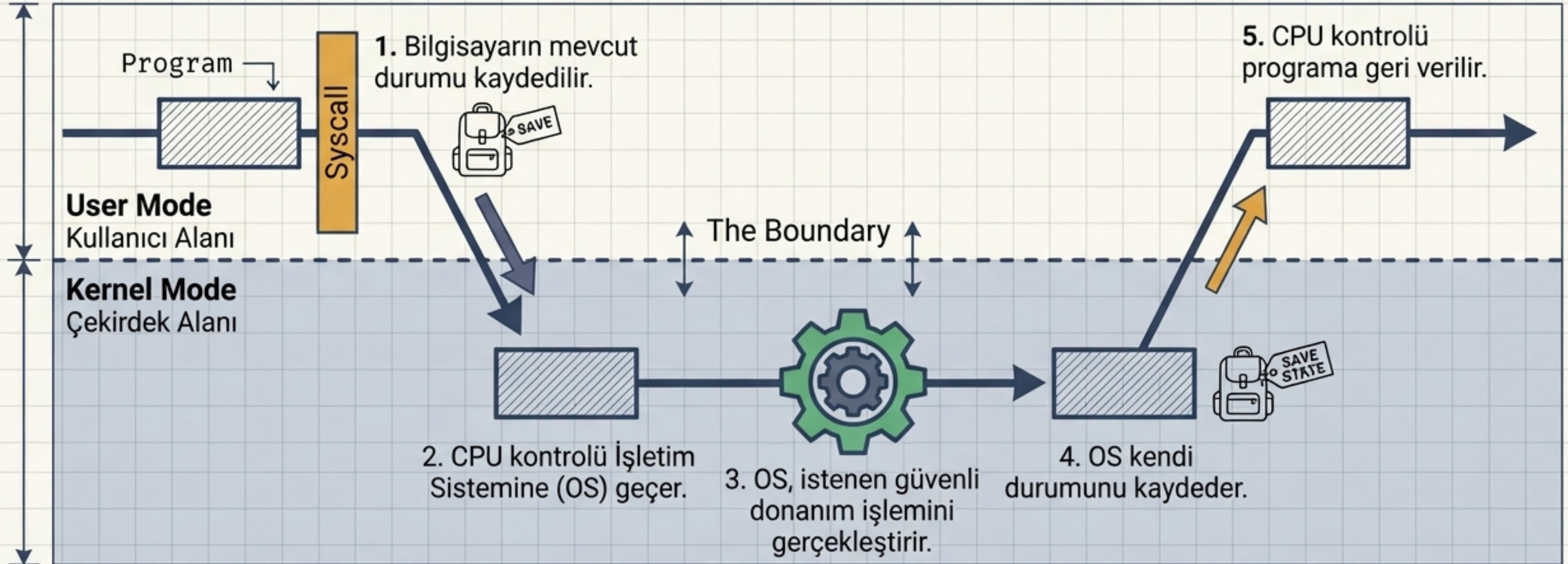
Sınır Kapısı: Fonksiyon vs. Sistem Çağrısı



	Fonksiyon	Sistem Çağrısı
Çalışma Alanı	Kullanıcı Alanı (User Mode).	Çekirdek Alanı (Kernel Mode).
Mekanizma	Bellekte başka bir prosedüre basit bir atlama.	Bağlam değişimi (Context Switch) ve işletim sistemine devir.
Hız	Çok hızlı (birkaç makine komutu).	Yavaş ve maliyetli.
Amaç	Kod tekrarını önlemek.	G/Ç ve donanım erişimi için güvenlik.

Bir Sistem Çağrısının Anatomisi (Bağlam Değişimi)

İşletim sisteminin G/Ç'yi kontrol etmesinin ana nedeni güvenlidir. Hatalı bir programın sistemi çökertmesini önlemek için CPU'nun yetkisi geçici olarak devredilmelidir.



Sınırdaki Hata Yönetimi: `errno`

Bir sistem çağrısı veya kütüphane fonksiyonu başarısız olduğunda, özel bir dönüş değeri (genellikle -1 veya NULL) verir. Hatanın asıl nedeni evrensel `errno` değişkenine atanır. `perror()` fonksiyonu bu kodu okunabilir bir mesaja çevirir.

perror and errno

```
int main()
{
    FILE *f;

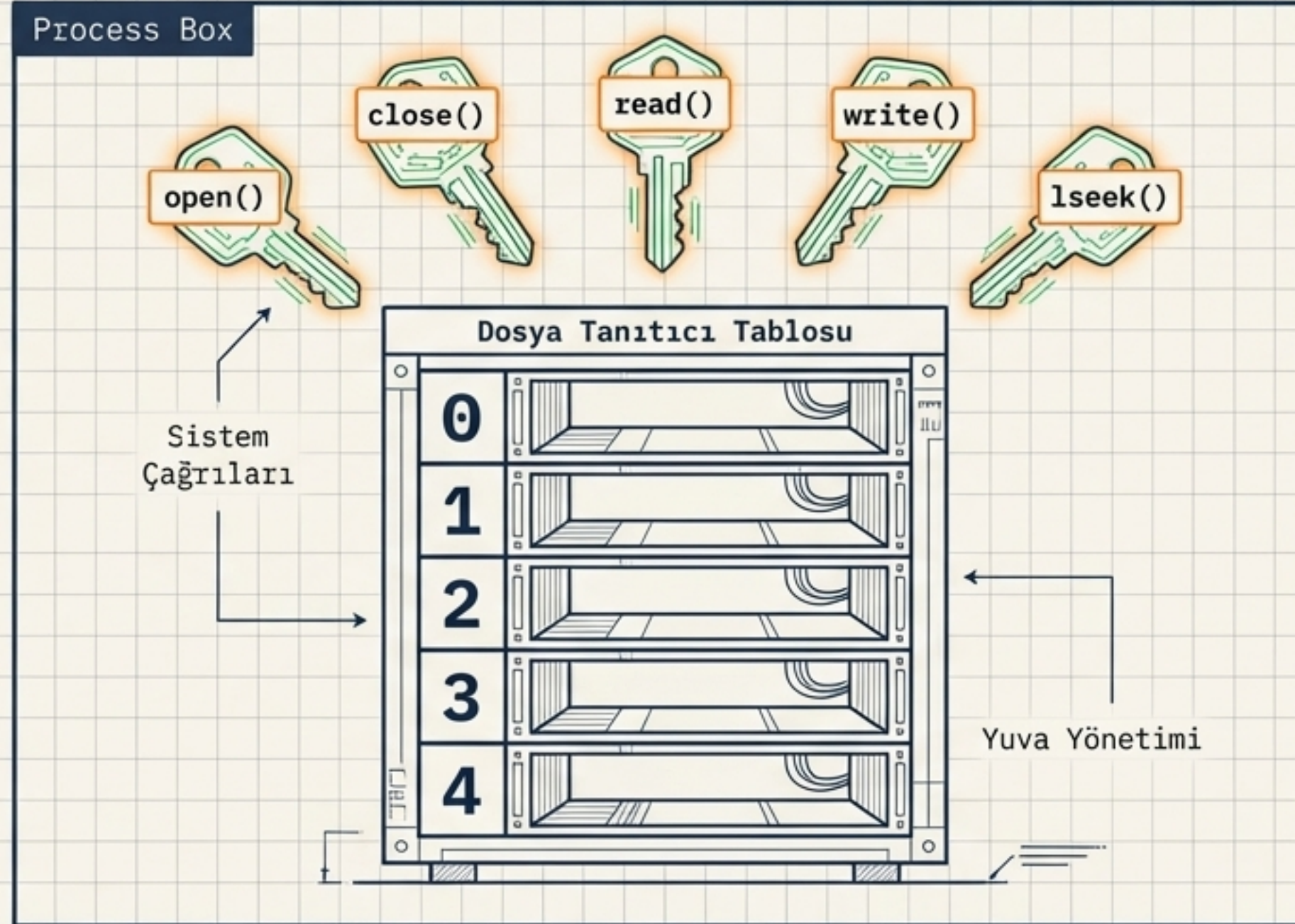
    f = fopen("/home/plank/noexist", "r");
    if (f == NULL) {
        printf("f = null. errno = %d\n", errno);
        perror("/home/plank/noexist");
    }
    return 0;
}
```

```
UNIX> echo "" > /home/plank/noexist
UNIX> chmod 0 /home/plank/noexist
UNIX> bin/ch1a
f = null.  errno = 13
/home/plank/noexist: Permission denied
```

```
UNIX> rm -f /home/plank/noexist
UNIX> bin/ch1a
f = null.  errno = 2
/home/plank/noexist: No such file or directory
UNIX>
```


Unix G/Ç İin 5 Temel Anahtar

İşletim sistemiyle doğrudan dosya alışverişi yapmak için bu 5 sistem çağrısı kullanılır. Tüm bu işlemler, "Dosya Tanıtıcı" (File Descriptor) adı verilen negatif olmayan tam sayılar (ints) üzerinden yürütölür.



1. `open()`: Dosya Tanıtıcıyı Talep Etmek

`open()` işletim sistemine bir dosyaya erişim isteği gönderir. İstek onaylanırsa, dosya tablosunda boş bir indeks olan 'Dosya Tanıtıcı' döndürür. Reddedilirse `-1` döner.

```
int open(const char *path, int flags, [ int mode ] );
```

```
/* This program opens the file "txt/in1.txt", and prints the  
return value of the open() system call. If the file exists, it  
will be a non-negative integer (three). */
```

```
#include <fcntl.h>  
#include <stdio.h>  
#include <stdlib.h>
```

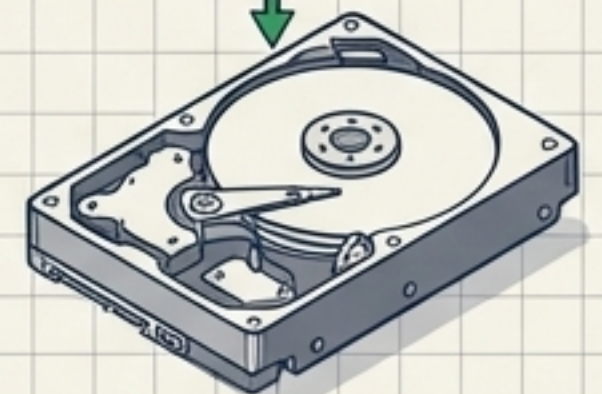
```
int main()  
{  
    int fd;  
  
    fd = open("txt/in1.txt", O_RDONLY);  
    printf("%d\n", fd);  
    return 0;  
}
```

```
UNIX> bin/o1  
3  
UNIX> mv txt/in1.txt tmp.txt  
UNIX> bin/o1  
-1  
UNIX> mv tmp.txt txt/in1.txt  
UNIX> bin/o1  
3  
UNIX> █
```

Dosya Tanıtıcı Tablosu

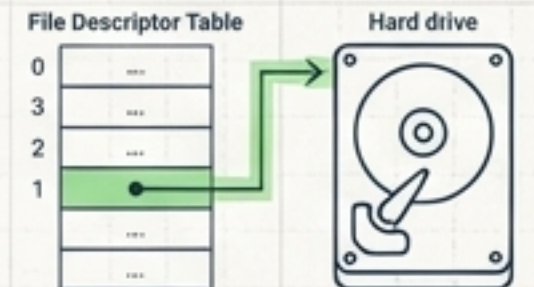
0	
1	
2	
3	

Yuva Yönetimi



`open()` Bayrakları (Flags) Karar Matrisi

	Dosya Zaten Var	Dosya Yok
O_WRONLY	Üzerine yazar. 	Hata verir (-1). 
O_WRONLY O_CREAT	Üzerine yazar, boyut küçülmez. 	Yeni dosya oluşturur. 
O_WRONLY O_CREAT O_TRUNC	Sıfır bayta kırpar (siler), yeniden yazar. 	Yeni dosya oluşturur. 



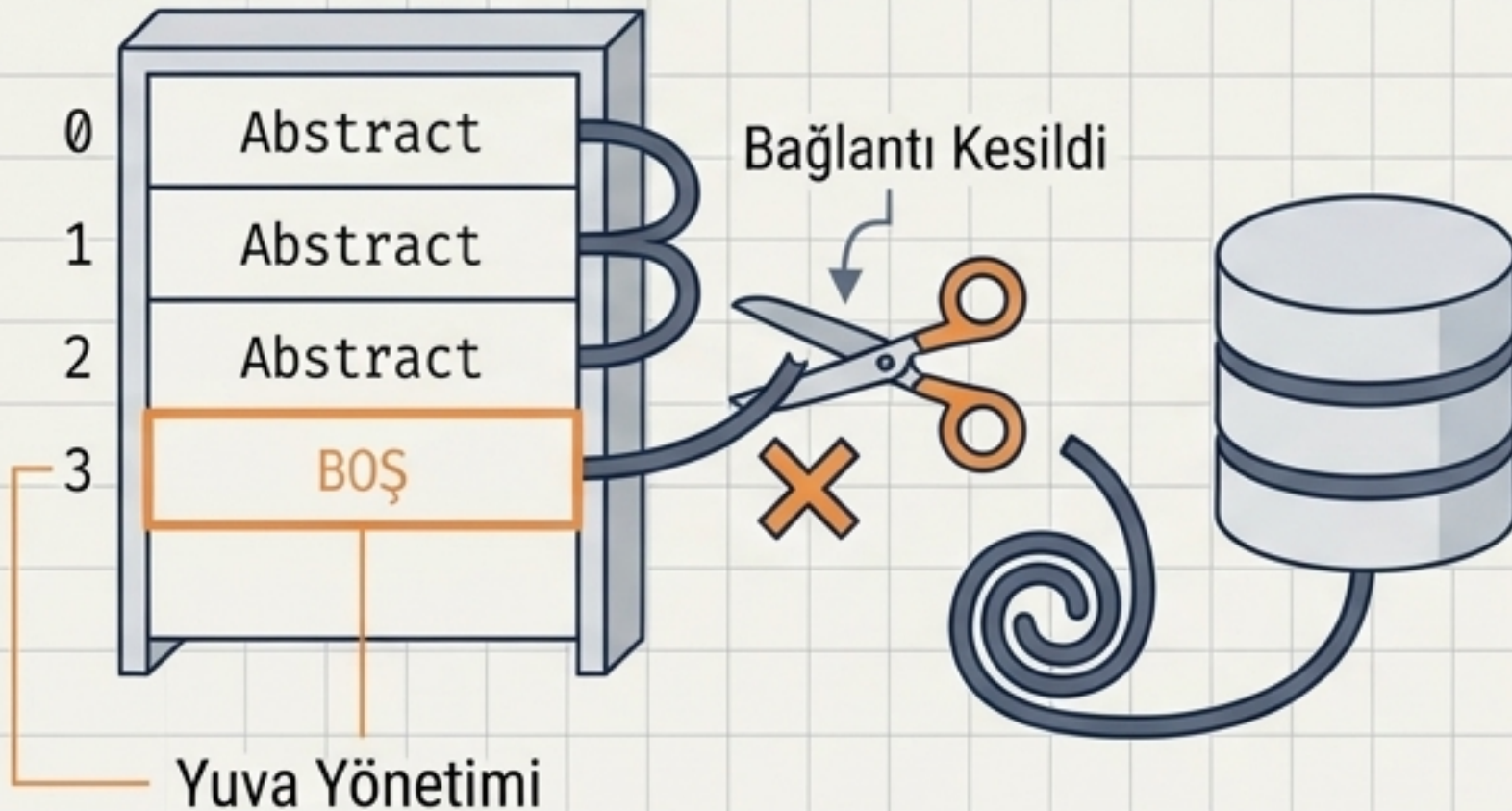
2. `close()`: İletişimi Sonlandırmak

`close()` işletim sistemine o dosya tanıtıcı ile işinizin bittiğini bildirir. Numara daha sonra tekrar kullanılabilir. Dosyayı kapatmadan defalarca açmak yasaldır ancak kaynakları tüketir. Önceden kapatılmış bir `fd`'yi tekrar kapatmak "Bad file descriptor" hatasına neden olur.

```
int close(int fd);
```

Anahtarın İadesi ve Kaynak Serbest Bırakma

'Dosya Tanıtıcı Tablosu'



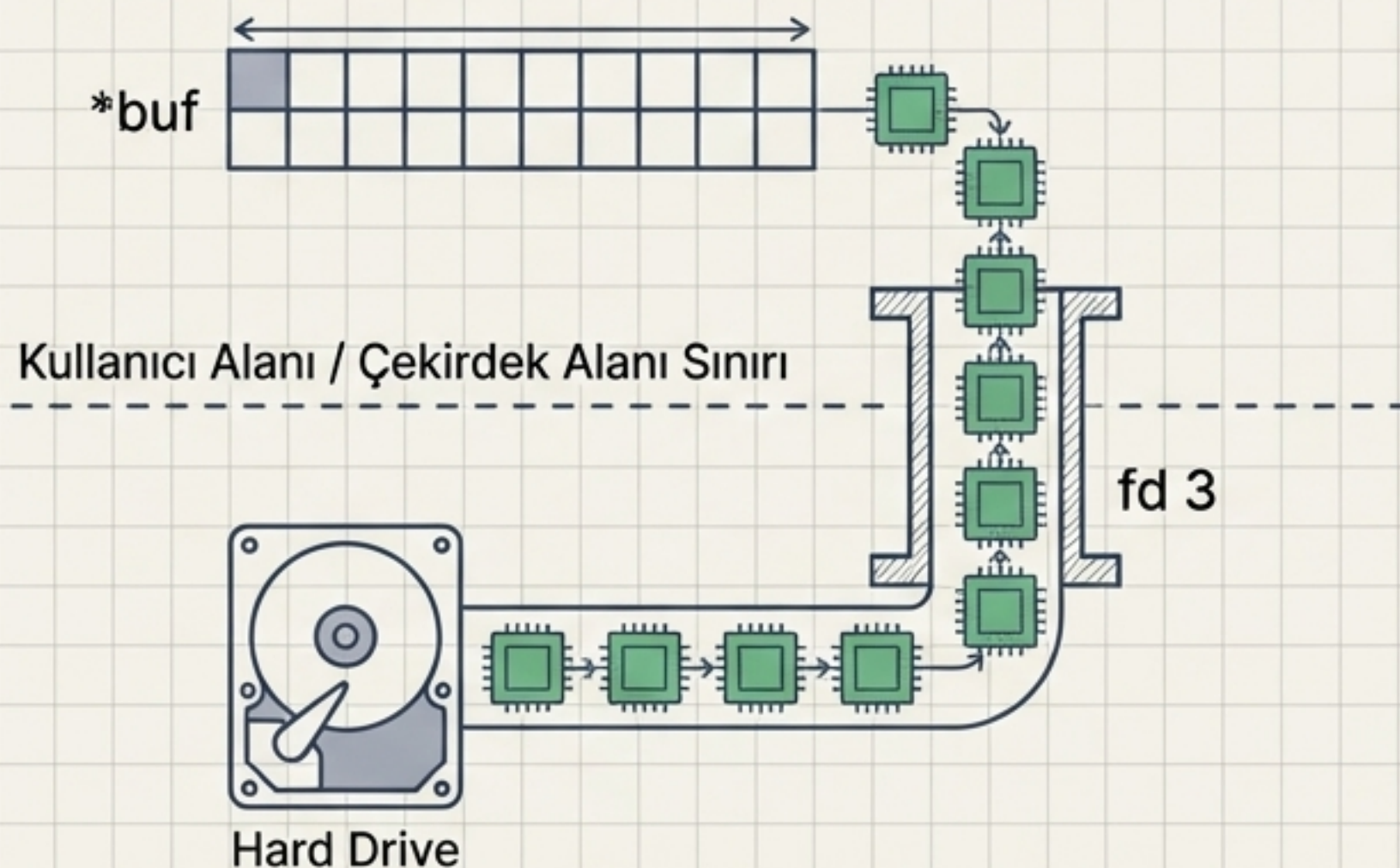
Hata Senaryosu: Önceden Kapatılmış Bir fd'yi Kapatmak

```
UNIX> bin/c1
Opened the file txt/in1.txt twice: Fd's are 3 and 4.
Closed both fd's.
Closed both fd's.
Reopened txt/in1.txt into fd2: 3.
Closed fd2. Now, calling close(fd2) again.
This should cause an error.
c1: Bad file descriptor
UNIX>
```


3. `read()`: Veriyi Kullanıcı Alanına Çekmek

İşletim sistemine fd ile belirtilen dosyadan belirli count kadar baytı okumasını ve bunları buf belleğine yerleştirmesini söyler. Okunan *gerçek* bayt sayısını döndürür. `0` dönmesi dosya sonuna (EOF) gelindiğini gösterir.

```
ssize_t read(int fd, void *buf, size_t count);
```



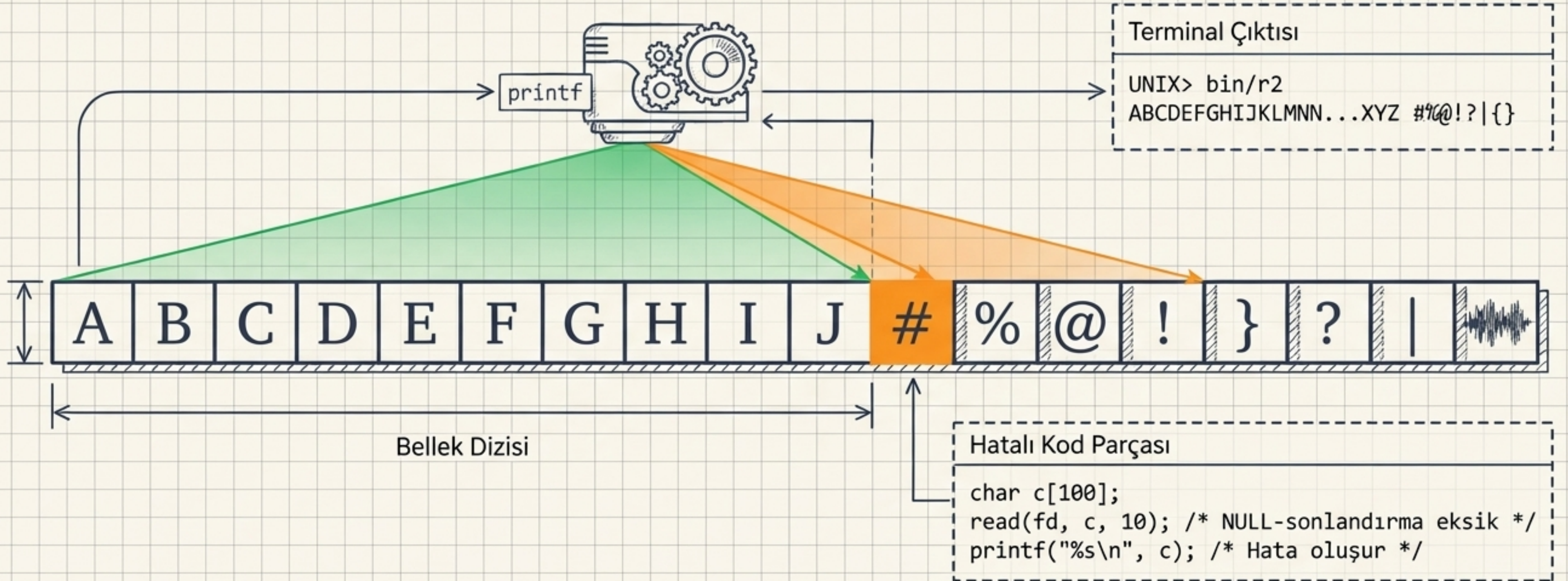
```
UNIX> bin/r1  
called read(3, c, 10). returned that 10  
bytes were read.
```

```
Those bytes are as follows:  
Jim Plank  
called read(3, c, 99). returned that 12  
bytes were read.
```

```
Those bytes are as follows:  
Claxton 221  
UNIX>
```


Kritik Uyarı: NULL Sonlandırıcı Eksikliği

Metin dosyalarında gizli bir NULL (`\0`) karakteri yoktur. `read()` çağrısı sadece diskteki ham baytları kopyalar. C dilinde bu baytları bir "string" olarak (`printf` ile) kullanmak istiyorsanız, okuma işleminden sonra son bayta manuel olarak `\0` eklemek zorundasınız.



4. `write()`: Çekirdeğe Veri Göndermek

`read()` çağrısının tam tersidir. `buf` belleğindeki baytları alır ve işletim sistemi aracılığıyla `fd`'ye **yazar**. Başarılı olursa yazılan gerçek bayt sayısını döndürür. Dosyanın `open()` sırasında nasıl bayraklandığı (0_TRUNC, 0_CREAT) yazma davranışını kökten değiştirir.

```
ssize_t write(int fd, const void *buf, size_t count);
```

```
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <string.h>
#include <stdlib.h>

int main()
{
    int fd, sz;

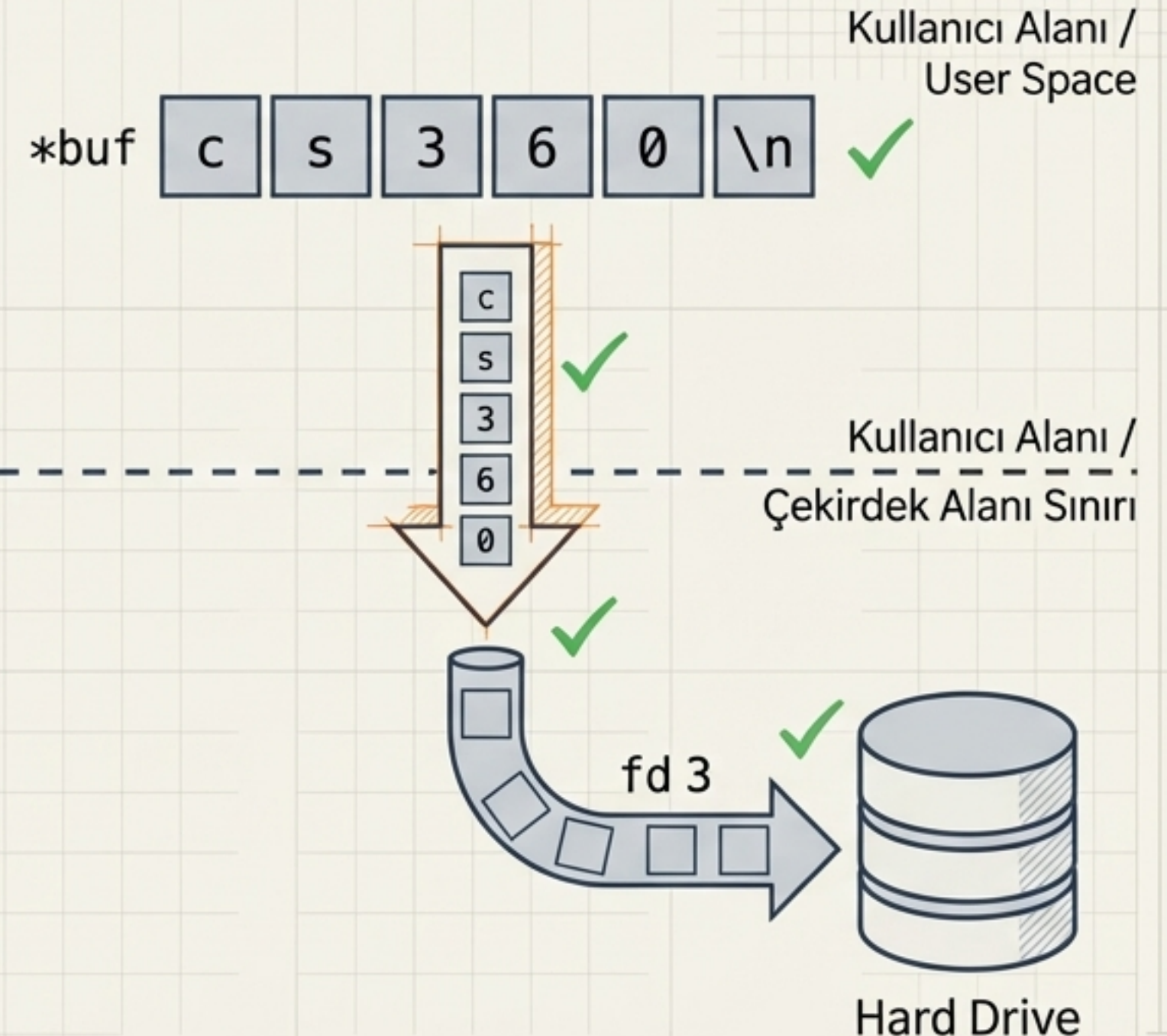
    fd = open("txt/out3.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (fd < 0) { perror("txt/out3.txt"); exit(1); }

    sz = write(fd, "cs360\n", strlen("cs360\n"));

    printf("called write(%d, \"cs360\n\", %ld). it returned %d\n",
        fd, strlen("cs360\n"), sz);

    close(fd);
    return 0;
}
```

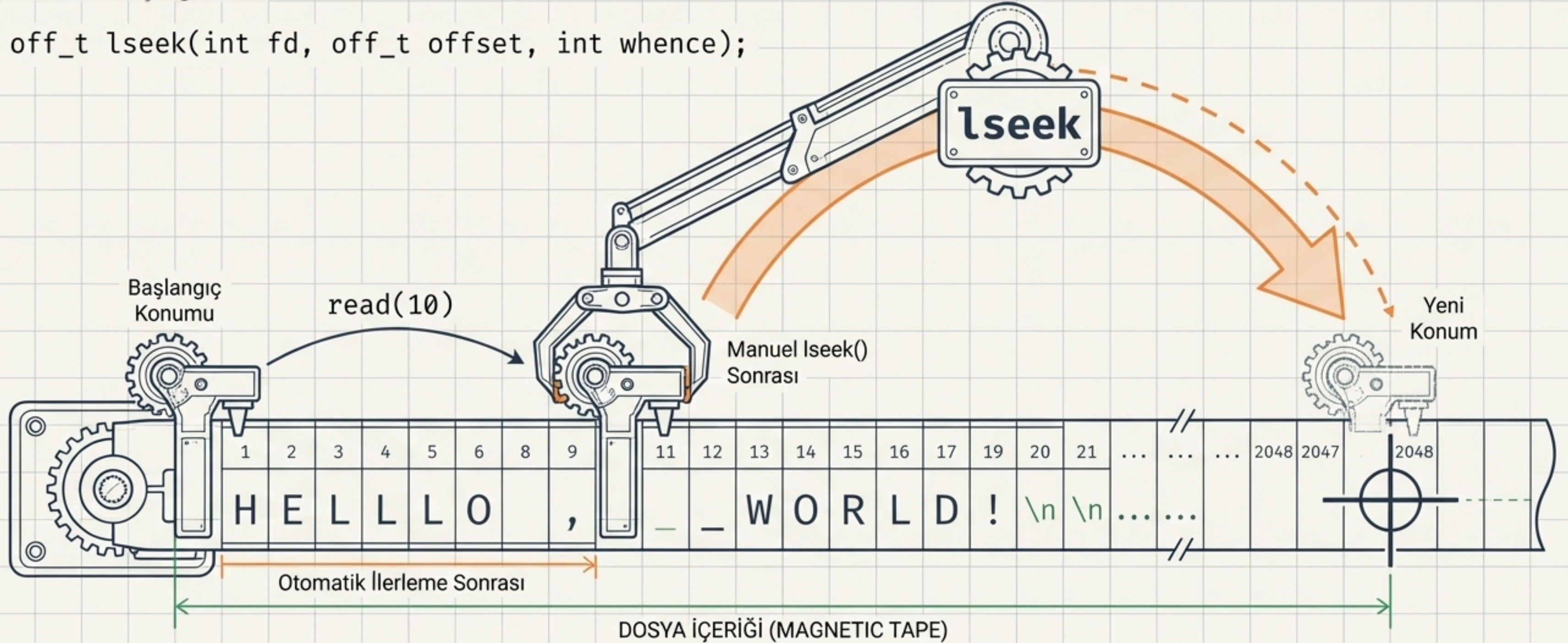
```
UNIX> bin/w1
called write(3, "cs360\n", 6). it returned 6
UNIX> cat txt/out3.txt
cs360
UNIX> █
```



5. `lseek()`: Dosya İşaretçisini (Playhead) Taşımak

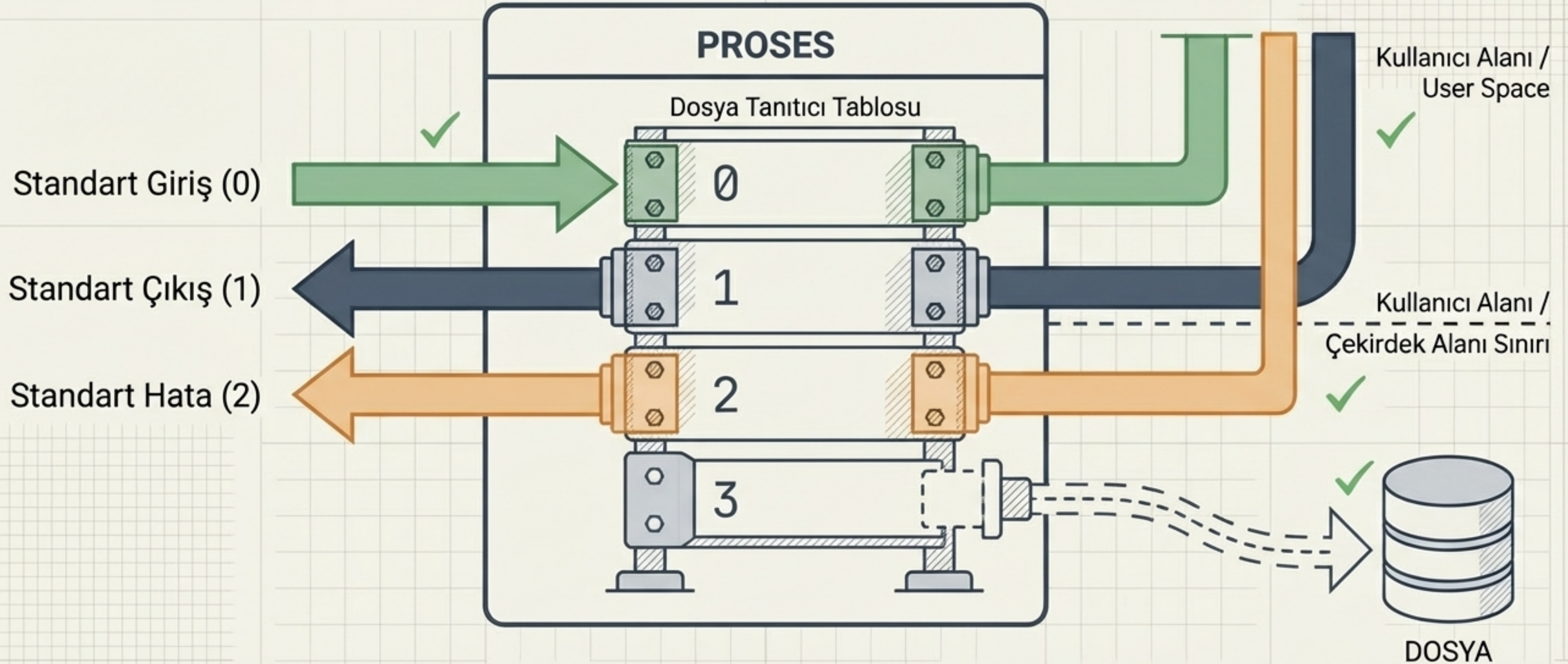
Açık olan her dosyanın bir "dosya işaretçisi" vardır. Okuma ve yazma işlemleri bu işaretçiyi otomatik ilerletir. `lseek()` bu işaretçiyi (baştan, mevcut konumdan veya sondan) manuel olarak hareket ettirmenizi sağlayan C Sistem Çağrısıdır.

```
off_t lseek(int fd, off_t offset, int whence);
```



Sentez: Standart G/Ç Boru Hattı

Unix'te her proses doğuştan üç açık dosya tanıtıcı ile gelir. İşletim sistemi bu üç anahtarı size otomatik olarak verir, böylece konsol ile anında iletişim kurabilirsiniz.



Aha! Anı: Her Şey Bir Dosya Tanıtıcıdır

Yıllardır kullandığınız standart C fonksiyonları aslında sadece sarf malzemesidir. Arka planda **printf** sadece **fd 1**'e bir **write()** çağrısıdır. Bu temel bilgiyle, donanım giriş ve çıkışlarını doğrudan işletim sistemi seviyesinde kendiniz kablolayabilirsiniz.

printf(...)

└───→ write(1, ...)

scanf(...)

└───→ read(0, ...)

```
#include <unistd.h>
```

```
int main()  
{
```

```
    char c;
```

```
    while (read(0, &c, 1) == 1) write(1, &c, 1);  
    return 0;
```

```
}
```

```
UNIX> bin/simpcat < txt/in1.txt
```

```
Jim Plank
```

```
Claxton 221
```

```
UNIX>
```


Son Görev: Çekirdeği Manuel Yönetmek

Şimdi sistem çağrılarını kullanarak kendi köprünüzü inşa edin.

Soru:

1. Konsoldan (Standart Giriş - fd 0) **read()** kullanarak veri alın.
2. Bu veriyi **open()** ile açtığınız **uyg1.txt** dosyasına **write()** ile yazın (Dosya yoksa **O_CREAT** kullanın).
3. **lseek()** ile başa dönün ve dosyadan tekrar **read()** ile okuyup, veriyi konsola (Standart Çıkış - fd 1 veya **printf** ile) geri yazdırın.